You said:

exec.pyc
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 37s

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

ChatGPT said:

Análise estática do arquivo `exec.pyc`

Classificação preliminar: malware

O arquivo analisado é um **bytecode Python compilado (`.pyc`)**, e não um arquivo executável nativo. Sua estrutura corresponde ao Python 3.13.

A inspeção inicial já revelou comportamentos altamente suspeitos: o programa instala serviços persistentes no Linux, executa varreduras de portas, tenta autenticar-se em servidores SSH usando senhas e possui um mecanismo de propagação para outros servidores.

A classificação preliminar é de malware, com fortes indícios de um worm que se propaga por SSH e utiliza força bruta de credenciais. A seguir, detalho os componentes encontrados e seus efeitos.

1. Identificação e estrutura

Propriedade	Resultado
Arquivo	`exec.pyc`
Formato	Bytecode Python
Versão identificada	Python 3.13
Tamanho	57.929 bytes
SHA-256	`e7c55ad35f526ef8213e68cf1618f62c74134dc3246b815b2850ef7d772a6212`
Tipo de análise	Estática
Execução durante a análise	Não realizada

O arquivo contém código compilado que pode ser inspecionado por meio da desserialização dos objetos de código Python e da desmontagem das instruções de bytecode.

Um aspecto importante: ele contém também código-fonte de outros componentes incorporado em suas constantes, permitindo reconstruir boa parte do funcionamento do programa.

2. Análise dos comportamentos encontrados

2.1. Instalação com privilégios de administrador

A função `main()` verifica se o programa está sendo executado com privilégios de root.

Em seguida, prepara os seguintes diretórios:

```
/usr/lib/.sysd-scan/
/usr/lib/.sysd-monitor/
/var/lib/.sysd-data/
/var/log/.sysd-monitor.log
```

Os nomes ocultos, iniciados por ponto, merecem atenção, especialmente porque são utilizados para armazenar componentes de um programa que pretende permanecer ativo no sistema.

Também instala dependências do sistema, como Python, Masscan, Screen e bibliotecas de comunicação SSH.

Comportamento suspeito

A preparação de toda essa infraestrutura não é simplesmente uma operação de instalação comum: ela viabiliza a execução contínua dos componentes de varredura e propagação.

2.2. Varredura de servidores na Internet

O código contém um script chamado `run_scan.sh`.

Ele utiliza o programa Masscan para identificar servidores com portas SSH abertas.

Os seguintes parâmetros foram encontrados:

```
PORTS="22,5522"
WORKERS=15
MAX_RATE=5000
```

O funcionamento é:

1. Obtém intervalos de endereços IP de um arquivo remoto.
2. Executa varreduras das portas 22 e 5522.
3. Extrai os endereços IP e portas encontrados.
4. Filtra os resultados já processados.
5. Encaminha os alvos para os módulos de autenticação SSH.

O processo é executado em um loop infinito, repetindo as varreduras continuamente.

Interpretação: o programa procura sistematicamente servidores SSH acessíveis para tentar obter acesso a eles.

2.3. Tentativa automatizada de autenticação SSH

Este é um dos indícios mais fortes de atividade maliciosa.

O arquivo contém um módulo denominado `ssh_scanner.py`.

Ele utiliza a biblioteca Paramiko para realizar conexões SSH e testar credenciais.

A função responsável pela tentativa de autenticação é `worker()`:

```
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

ssh.connect(
    hostname=ip,
    port=port,
    username=username,
    password=password,
    timeout=SSH_TIMEOUT
)
```

O usuário padrão é:

```
root
```

O programa também contém duas senhas embutidas no código e realiza testes independentes utilizando cada uma delas.

Quando consegue autenticação, registra o endereço do servidor em arquivos de resultados.

Os arquivos utilizados são:

```
/var/lib/.sysd-data/LeitboGi0ro.txt
/var/lib/.sysd-data/q123.txt
```

Isso demonstra uma atividade automatizada de tentativa de credenciais em múltiplos servidores.

2.4. Propagação para outros servidores

Evidência decisiva de comportamento de worm

O código incorporado em `MONITOR_SCRIPT_CONTENT` apresenta um módulo chamado `monitor_script.py`.

Sua função é monitorar os resultados encontrados pelo scanner e tentar obter acesso aos servidores identificados.

Quando consegue autenticação SSH:

- Abre uma conexão SFTP.
- Copia o próprio código para o servidor remoto.
- Salva o arquivo em `/tmp/bendi.py`.
- Verifica e instala Python 3, quando necessário.
- Executa o código copiado.
- Remove o arquivo temporário após a execução.

O trecho essencial é:

```
REMOTE_PAYLOAD_PATH = "/tmp/" + PAYLOAD_SCRIPT_FILENAME

sftp = ssh_client.open_sftp()

with sftp.file(REMOTE_PAYLOAD_PATH, 'w') as f:
    f.write(payload_content_to_upload)
```

Em seguida:

```
success, output = execute_remote_command(
    ssh_client,
    "python3 " + REMOTE_PAYLOAD_PATH
)

execute_remote_command(
    ssh_client,
    "rm " + REMOTE_PAYLOAD_PATH
)
```

O código propagado instala novamente a infraestrutura de varredura e monitoramento no servidor remoto.

Portanto, cada servidor comprometido pode se tornar uma nova fonte de propagação.

Esse comportamento caracteriza um **worm (verme de computador)**.

2.5. Comunicação com infraestrutura externa

Foram identificados os seguintes endereços:

Endereço	Finalidade observada
`sou.pp.ua`	API central de coordenação
`del.sou.pp.ua`	Distribuição de arquivos de configuração e instaladores
`api.telegram.org`	Envio de notificações ao operador

A API central possui dois endpoints:

```
/check_ip.php
/add_ip.php
```

Eles são utilizados para verificar e registrar os servidores já processados.

O programa também utiliza a API do Telegram para enviar relatórios.

Um detalhe especialmente preocupante: as mensagens contêm informações sobre os servidores acessados, os resultados das operações e a credencial utilizada na autenticação.

O bytecode contém um token de bot e um identificador de conversa do Telegram codificados diretamente no programa.

Isso representa um canal de comunicação e exposição de informações ao operador da infraestrutura.

2.6. Persistência no sistema operacional

O programa cria dois serviços systemd:

```
scan-runner.service
file-monitor.service
```

Ambos apresentam configurações que incluem:

```
User=root
Restart=always
RestartSec=10
```

Também executa:

```
systemctl daemon-reload
systemctl enable scan-runner.service
systemctl restart scan-runner.service

systemctl enable file-monitor.service
systemctl restart file-monitor.service
```

Isso garante que os componentes sejam iniciados automaticamente e reiniciados em caso de falha.

Além disso, após concluir a instalação, o programa agenda a exclusão do próprio arquivo executado:

```
sleep 2 && rm -- 'caminho_do_script'
```

A exclusão do instalador dificulta a investigação posterior e é outro indicador suspeito.

2.7. Download e execução de componentes externos

Também encontrei downloads de:

```
https://del.sou.pp.ua/install.sh
https://del.sou.pp.ua/install_tm.sh
```

O primeiro é executado com Bash depois do download.

O segundo também é executado com Bash, acompanhado de um identificador de instalação.

O programa não implementa verificação criptográfica de integridade desses scripts.

Não foi possível determinar o comportamento interno desses arquivos externos apenas pela análise do `exec.pyc`, pois seus conteúdos não estão incorporados integralmente no bytecode.

3. Indicadores de comprometimento (IOCs)

Os seguintes indicadores podem ser utilizados em uma investigação de segurança:

Categoria	Indicador
Diretório	`/usr/lib/.sysd-scan/`
Diretório	`/usr/lib/.sysd-monitor/`
Diretório	`/var/lib/.sysd-data/`
Log	`/var/log/.sysd-monitor.log`
Serviço	`scan-runner.service`
Serviço	`file-monitor.service`
Arquivo temporário	`/tmp/bendi.py`
Domínio	`sou.pp.ua`
Domínio	`del.sou.pp.ua`
Protocolo	SSH
Portas examinadas	TCP 22 e 5522
Comunicação externa	API do Telegram

Esses indicadores devem ser considerados em conjunto, pois alguns elementos isolados, como serviços systemd ou conexões SSH, também podem aparecer em programas legítimos.

4. Classificação técnica

Veredito final MALWARE

Worm SSH

Categoria principal: Worm (verme de computador).

Subcategoria: Worm com propagação por SSH e tentativas automatizadas de autenticação (*password spraying*).

Características secundárias:

- Varredura automatizada de portas e servidores.

- Propagação lateral por credenciais SSH.
- Persistência por serviços systemd.
- Comunicação e relatórios via Telegram.
- Coordenação por infraestrutura central.
- Exclusão do instalador após a implantação.

Por que não é apenas uma ferramenta administrativa?

Uma ferramenta legítima de monitoramento ou administração poderia utilizar SSH, systemd, Paramiko e até mesmo Telegram.

Entretanto, a combinação encontrada neste arquivo é determinante:

1. Procura indiscriminadamente servidores SSH em intervalos de IP.
2. Testa senhas predefinidas contra servidores encontrados.
3. Registra autenticações bem-sucedidas.
4. Instala automaticamente seu próprio código nos servidores acessados.
5. Repete o processo nos servidores comprometidos.
6. Mantém os componentes ativos mediante serviços persistentes.
7. Comunica os resultados a uma infraestrutura externa.

Essa combinação demonstra comportamento de autorreplicação e comprometimento automatizado de outros sistemas.

5. Conclusão

MALWARE CONFIRMADO POR ANÁLISE ESTÁTICA

O arquivo `exec.pyc` é malicioso.

Sua classificação principal é **SSH Worm**, pois possui mecanismos explícitos para identificar servidores vulneráveis a credenciais conhecidas, comprometer esses servidores e propagar automaticamente sua própria carga maliciosa.

A análise foi realizada sem executar o arquivo. Portanto, os comportamentos descritos são funcionalidades implementadas no código; não estou afirmando que houve uma infecção efetiva durante esta análise.

Recomendação: não execute esse arquivo. Caso ele já tenha sido executado em um servidor, considere o sistema comprometido, revogue as credenciais SSH expostas, invalide o token do Telegram identificado e investigue os dois serviços systemd. Para

servidores comprometidos com privilégios de root, a reinstalação a partir de uma imagem confiável é a alternativa mais segura.

‹/›

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal é **Worm (verme de computador)**, mais especificamente um **SSH Worm**.

Ele se propaga automaticamente pela rede, varrendo servidores SSH, tentando credenciais e copiando seu próprio código para máquinas comprometidas.